Introduction to MPI

Goal of MPI

MPI is intended for:

- programs running on **distributed-memory systems**,
- programs that need **explicit communication** between processes,
- and programs that must remain **portable and efficient** across platforms.

*MPI is **not a programming language** - It is a **library** that is called from languages such as C, C++, and Fortran.*

Basic MPI Concepts

- Process
- Rank
- Size
- Communicator

Basic MPI Concepts

Process

A process is an independent execution unit with its own private memory.
In MPI, a parallel program consists of multiple processes that run simultaneously and cooperate by exchanging messages.

Rank

The rank is a unique integer identifier assigned to each process within a communicator.
Ranks are used to distinguish processes and to decide which process performs which part of the work.

Size

The size is the total number of processes participating in a communicator.
It tells each process **how many processes exist in the parallel program**.

Basic MPI Concepts

Communicator

A communicator combines a **group of processes** with a **communication context**.
All MPI communication operations are performed within a communicator, with `MPI_COMM_WORLD` being the default communicator containing all processes.

The six-function version of MPI

`MPI_Init`	Initialize MPI
`MPI_Comm_size`	Find out how many processes there are
`MPI_Comm_rank`	Find out which process I am
`MPI_Send`	Send a message
`MPI_Recv`	Receive a message
`MPI_Finalize`	Terminate MPI

MPI Datatypes

- MPI_INT
- MPI_FLOAT
- MPI_DOUBLE

MPI_Init

MPI_Init(int *argc, char ***argv)

- Must be the **first MPI call** in the program.
- Must be called by **all processes**.
- Initializes internal data structures used by MPI.
- Allows MPI to process command-line arguments if needed.

No MPI routine may be called before `MPI_Init`.

MPI_Finalize()

MPI_Finalize()

- Must be the **last MPI call** in the program.
- Ensures all MPI operations have completed.
- Frees resources used by MPI.

MPI_Comm_rank

MPI_Comm_rank(MPI_Comm comm, int *rank)

Determines the **rank** of the calling process.

- `comm`: communicator in which the rank is defined
- `rank`: output parameter storing the process ID

*Each process receives a **unique rank** within the communicator.*

MPI_Comm_size

MPI_Comm_size (MPI_Comm comm, int *size)

Determines the **number of processes** in a communicator.

- `comm`: communicator being queried
- `size`: output parameter storing total process count

*All processes in the communicator obtain the **same value of size**.*

MPI_Send

MPI_Send (void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm)

- **buf**: starting address of the data to send
- **count**: number of data elements
- **datatype**: type of each data element
- **dest**: rank of the destination process
- **tag**: integer message identifier
- **comm**: communicator defining the communication context

The send operation completes when the send buffer can be safely reused.

MPI_Recv

MPI_Recv (void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)

- **buf**: starting address of the receive buffer
- **count**: maximum number of elements to receive
- **datatype**: type of expected data
- **source**: rank of the sending process (or `MPI_ANY_SOURCE`)
- **tag**: expected message tag (or `MPI_ANY_TAG`)
- **comm**: communicator defining the communication context
- **status**: structure containing information about the received message

MPI_Bcast

MPI_Bcast(void *buf, int count, MPI_Datatype datatype, int root, MPI_Comm comm)

- **buf – Address** of the data buffer.
  - On the **root process**, it contains the data to be sent.
  - On **all other processes**, it receives the data.
- **count – Number** of elements to broadcast.
- **datatype – Type** of each data element (e.g., `MPI_INT`, `MPI_DOUBLE`).
- **root -** Rank of the process that initially holds the data.
- **comm -** Communicator that defines the group of participating processes.

Why Measure Time in Parallel Programs?

In parallel programs, performance depends not only on computation, but also on **communication and synchronisation,** making accurate measurement of **elapsed wall-clock time** essential for evaluating and comparing parallel implementations.

double MPI_Wtime(void)

MPI provides `MPI_Wtime` as a **portable, high-resolution wall-clock timer**.

It returns the current time in seconds and is intended to be used by taking the difference between two calls.

Example - Matrix Vector Multiplication Problem

Matrix–vector multiplication computes a vector **y** from a matrix **A** and a vector **x**:

$$y = A \times x$$

Here:

- **A** is a matrix with many rows and columns.
- **x** is a vector,
- **y** is the resulting output vector.

$$y_i = \sum_j A_{ij} \times x_j$$

How Is the Data Distributed Across Processes?

- The **rows of the matrix A** are distributed across processes.
- Each process is responsible for computing **one or more rows** of the result vector **y**.
- The **vector x** is needed by **every process**, because each row computation requires the full vector.

This leads to a natural separation of data:

Matrix rows → distributed,
Vector x → shared.

Manager–Worker (Self-Scheduling) Approach

In this approach:

- One process acts as a **manager**.
- The remaining processes act as **workers**.
- Work (matrix rows) is assigned dynamically, not in advance.

This design allows:

- Better **load balancing** when processes run at different speeds.
- Faster processes to receive more work.
- Slower processes to receive less work.

The example is chosen to demonstrate how MPI programs can dynamically distribute work using message passing, rather than relying on fixed data partitioning.

Three Logical Parts of the Parallel Solution

Manager Process

- Controls the execution
- Distributes work to workers
- Collects computed results

Worker Processes

- Receive work from the manager
- Perform the computation for assigned rows
- Send results back to the manager

Communication Mechanism

- Transfers data between manager and workers
- Ensures correct coordination and completion

Role of the Manager Process

- Distributes the required shared data to all workers.
- Assigns rows of the matrix to worker processes for computation.
- Receives partial results from workers.
- Stores each result in the correct position of the output vector.

Role of the Worker Processes

- Receives a row of the matrix from the manager.
- Computes the dot product of that row with the vector.
- Sends the computed result back to the manager.

Communication Pattern in the Program

- Broadcasting the input vector so that all worker processes receive the same data.
- Sending matrix rows from the manager to individual workers.
- Sending computed results from workers back to the manager.

Question

1. Starting from the self-scheduling matrix–vector multiplication program discussed in class, first modify the program so that each worker process is assigned a fixed set of rows at the beginning and no dynamic reassignment of work takes place. Compare this static version with the original self-scheduling version and comment on the effect on load balance and idle time.

2. Next, modify the original self-scheduling program so that the manager sends multiple rows at a time to each worker instead of a single row. Study how increasing the number of rows per message affects the number of messages, communication overhead, and overall efficiency. Compare your observations with the original implementation and discuss the choice of an optimal value.

How To Access LiTE-Pi

Username - ***yourname@172.28.8.3*** (lowercase)

Password - ***entrynumber*** (lowercase)

Change your password using - ***passwd***

THANK YOU